

Predictive Modeling of Obesity Levels Using Lifestyle and Dietary Data

PH 3022 – Machine Learning and Neural
Computation

Group Members

H. S. D. Hewanadugala- S16802

R.M.K.H. Gamhatha – S16851

D.M.V. Anjana – S16844

W.S.M. Botheju – S16624

Table of Content

Contents

1. Abstract	4
2. Introduction	5
3. Methodology	6
3.1 Description of the Attributes in Dataset	6
3.2 Graphical Presentation of the data	8
3.3 Data Acquisition and Preprocessing	11
3.3.1 Data Loading and Initial Inspection	11
3.3.2 Duplicate Removal	11
3.3.3 Feature Engineering (BMI Calculation)	11
3.3.4 Encoding of Categorical Variables	11
3.3.5 Handling missing values with KNN Imputation	12
3.3.6 Target Variable Encoding	13
3.4 Model Architecture and Building	13
3.4.1 Random Forest design- by Samodani	13
3.4.2 Neural Network Design - by Kavindu	16
3.4.3 Support Vector Machine Design - by Malinda	18
3.4.4 CatBoost Model – by Vishwa	20
4 Data Analysis	22
4.1 Random Forest Model	22
4.1.1 Best hyperparameters	22
4.1.2 Model Performance	23
4.1.3 Classification Report	24
4.1.4 Overfitting Analysis	24
4.1.5 Cross Validation Score	25
4.1.6 Confusion matrix	26
4.2 Neural Network Design	27
4.2.1 Model Performance and Results	27
4.2.2 Classification Metrics	27
4.2.3 Error Analysis and Discussion	28
4.2.4 Analysis of Confusion Matrix	29

4.2.5 Impact of Feature Engineering (BMI).....	29
4.2.6 Class Weighting Effectiveness.....	30
4.3 CatBoostClassifier	30
4.3.1 Model Performance and Results.....	30
4.3.2 Classification Report	31
4.3.3 Confusion Matrix Heatmap.....	32
4.4 Support Vector Machine	33
4.4.1 Model Performance and Results.....	33
4.4.2 Cross validation Performance	33
4.4.3 Distribution of Target Variable Classes	33
4.4.4 Classification report	34
4.4.5 Heatmap of the final output predictions	35
5. Discussion	36
6. Conclusion	37
7. References.....	38
8. Appendix.....	39

1. Abstract

A machine learning framework was developed to predict obesity levels based on eating habits, physical activity, and personal data. The dataset contained information such as gender, age, height, weight, family history of overweight, vegetable intake, meal frequency, consumption of high-calorie food (FAVC), water consumption, physical activity frequency (FAF), technology use (TUE), transportation method (MTRANS), and other lifestyle factors. The target variable was the obesity level (e.g., Insufficient Weight, Normal Weight, Overweight Level I, Overweight Level II, Obesity Type I, Obesity Type II, Obesity Type III).

The dataset underwent a structured preprocessing pipeline. First, duplicate records were removed to improve data quality. A new feature, Body Mass Index (BMI), was engineered from height and weight, along with additional interaction features to enhance predictive performance. Categorical variables were then encoded using appropriate strategies: ordinal encoding for ordered categories (CAEC and CALC), label encoding for binary variables (Gender, family history, FAVC, SMOKE, SCC), and one-hot encoding for the transportation method (MTRANS). The target variable was manually encoded into numerical labels. After encoding, the dataset was split into training (80%) and testing (20%) sets using stratified sampling to preserve class distribution. Missing values were handled using K-Nearest Neighbors (KNN) imputation with 5 neighbors, which fills missing values by identifying similar data points and preserving relationships between features. Finally, feature scaling was applied using StandardScaler to normalize the data before model training.

Four different machine learning models were built and tested by four team members: Random Forest, CatBoost, Deep Neural Network, and Support Vector Machine (SVM). Each model was improved by tuning its hyperparameters using methods such as RandomizedSearchCV, GridSearchCV, Bayesian Optimization, or Keras Tuner with the Hyperband algorithm.

Among the four models, the Random Forest achieved strong performance with a testing accuracy of approximately 95% and a macro F1-score above 0.94, demonstrating balanced performance across all obesity classes. The most important factors for predicting obesity included family history of overweight, frequency of high-calorie food consumption (FAVC), vegetable intake (FCVC), physical activity frequency (FAF), and transportation type (MTRANS). This model can help health professionals and individuals understand lifestyle factors that contribute to obesity and support early prevention efforts.

2.Introduction

Obesity is a growing health problem worldwide that occurs when a person has too much body fat, leading to serious issues like diabetes, heart disease, high blood pressure, and joint problems. In Sri Lanka, obesity has become a major concern due to changing lifestyles, including more consumption of high-calorie fast food, less physical activity, and increased use of cars and motorbikes instead of walking. Common causes of obesity include eating high-calorie foods frequently, low vegetable intake, skipping meals, drinking too little water, spending too much time on computers or watching TV, and having a family history of being overweight. Doctors classify obesity into seven levels: Insufficient Weight, Normal Weight, Overweight Level I, Overweight Level II, Obesity Type I, Obesity Type II, and Obesity Type III , each with different health risks and treatment needs.

In this project, we aim to build a machine learning model that can predict a person's obesity level based on their eating habits, physical activity, and personal information such as age, gender, height, weight, and transportation method. The dataset goes through a careful preparation process before training. This includes splitting data into training (80%) and testing (20%) sets to prevent data leakage, creating a new BMI feature from height and weight, converting text categories into numbers using different encoding methods (ordinal, label, and one-hot encoding), and handling missing values using K-Nearest Neighbors (KNN) imputation with 5 neighbors. Four different machine learning models are built and tested by four team members: Random Forest, Deep Neural Network, Support Vector Machine (SVM), and CatBoost. Each model is tuned using methods like RandomizedSearchCV, GridSearchCV, Bayesian Optimization or Keras Tuner with the Hyperband algorithm to find the best settings for accurate predictions.

The final model can help health professionals and individuals understand which lifestyle factors most affect obesity. By identifying at-risk people early, we can encourage healthier eating habits, more physical activity, better water consumption, and reduced screen time before serious health problems develop. This project also shows how machine learning can be used in healthcare to support prevention strategies and improve public health outcomes.

3. Methodology

3.1 Description of the Attributes in Dataset

Before building any machine learning models, it is important to understand the features (input variables) and the target variable (what we want to predict). The dataset contains information about individuals' eating habits, physical activity levels, and personal details. A brief description of each variable is shown in the table below.

Column Name	Description
Gender	Sex of the individual (Male or Female)
Age	Age of the individual in years
Height	Height of the individual in meters
Weight	Weight of the individual in kilograms
family_history_with_overweight	Whether the individual has a family member who is overweight (yes or no)
FAVC	Frequency of consuming high calorie food (yes or no)
FCVC	Frequency of eating vegetable (numeric scale 1= Never, 2=Sometimes, 3=Always)
NCP	Number of main meals per day
CAEC	Consumption of food between meals (no, sometimes, Frequently, Always)
SMOKE	Whether the individual smokes (yes or no)
CH2O	Daily water consumption in liters
SCC	Calories consumption in liters

FAF	Physical activity frequency per week (numeric scale)
TUE	Time spent using technology devices per day
CALC	Alcohol consumption frequency (no, Sometimes, Frequently, Always)
MTANS	Main transportation method used
NObeyesdad	Target variable – Obesity level of the individual

The target variable, NObeyesdad has seven possible classes,

1. Insufficient Weight
2. Normal Weight
3. Overweight Level I
4. Overweight Level II
5. Obesity Type I
6. Obesity Type II
7. Obesity Type III

These classes represent different levels of obesity, ranging from being underweight to severely obese. The ability to accurately predict the obesity level based on eating habits, physical activity, and personal information is the main goal of this machine learning project.

In addition to the lifestyle factors mentioned above, the dataset contains missing values in some columns. Features such as Height, Weight, FCVC, and others may have gaps due to incomplete data collection. Addressing these missing values using K-Nearest Neighbors (KNN) imputation, is essential to ensure the dataset is complete and the models are reliable.

Overall, the dataset provides a rich set of features related to diet, exercise, daily habits, and personal background. This makes it a strong foundation for building machine learning models that can predict obesity levels and help identify lifestyle factors that contribute to weight problems.

3.2 Graphical Presentation of the data

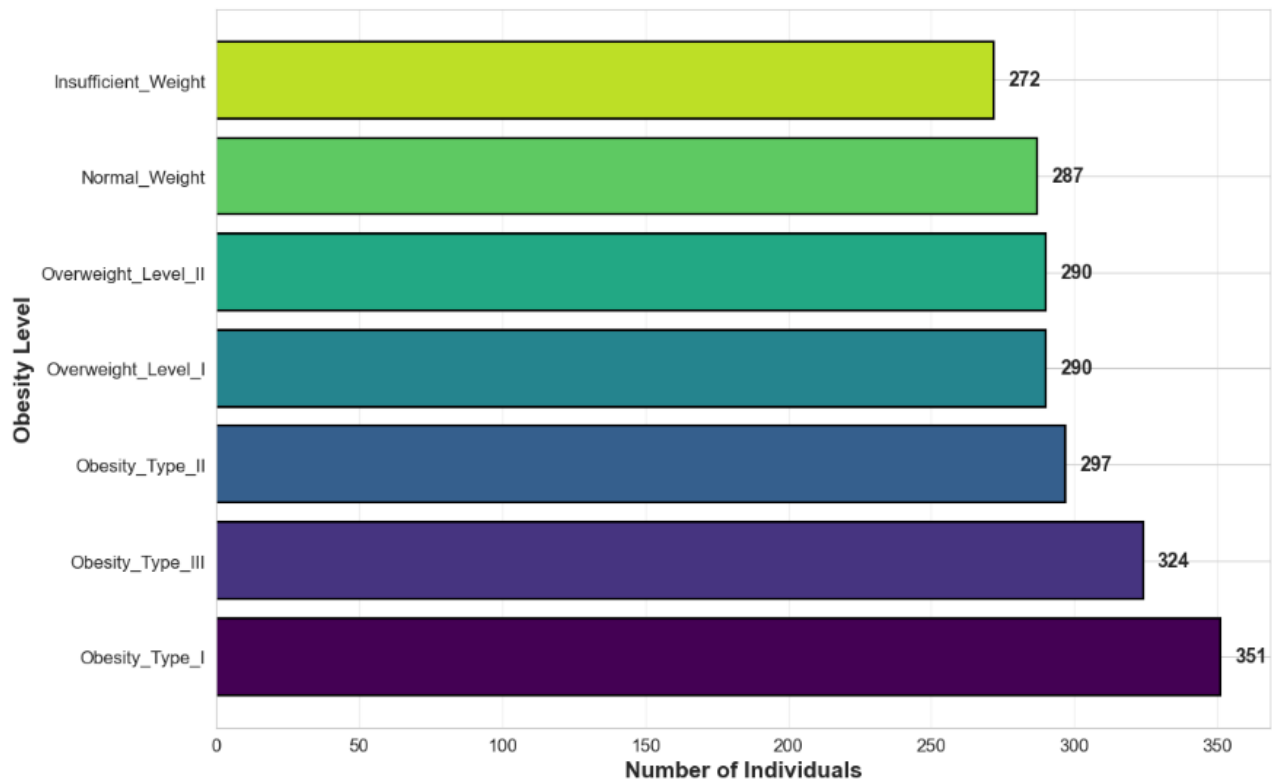


Figure 1 : Distribution of Obesity Levels in the dataset

This horizontal bar chart shows the distribution of samples across different obesity categories in the dataset. Each bar represents a specific class (such as Normal Weight, Overweight Level I & II, and Obesity Types I–III), while the numbers at the end of the bars indicate how many individuals belong to each category. The counts are very close to each other (around 287–297 samples per class), which means the dataset is well-balanced with no major class dominating the others. This balance is important because it helps the model learn each category fairly and reduces bias toward any single class. Overall, the plot confirms that the dataset has an even representation of obesity levels, which contributes to the strong and reliable performance of the model.

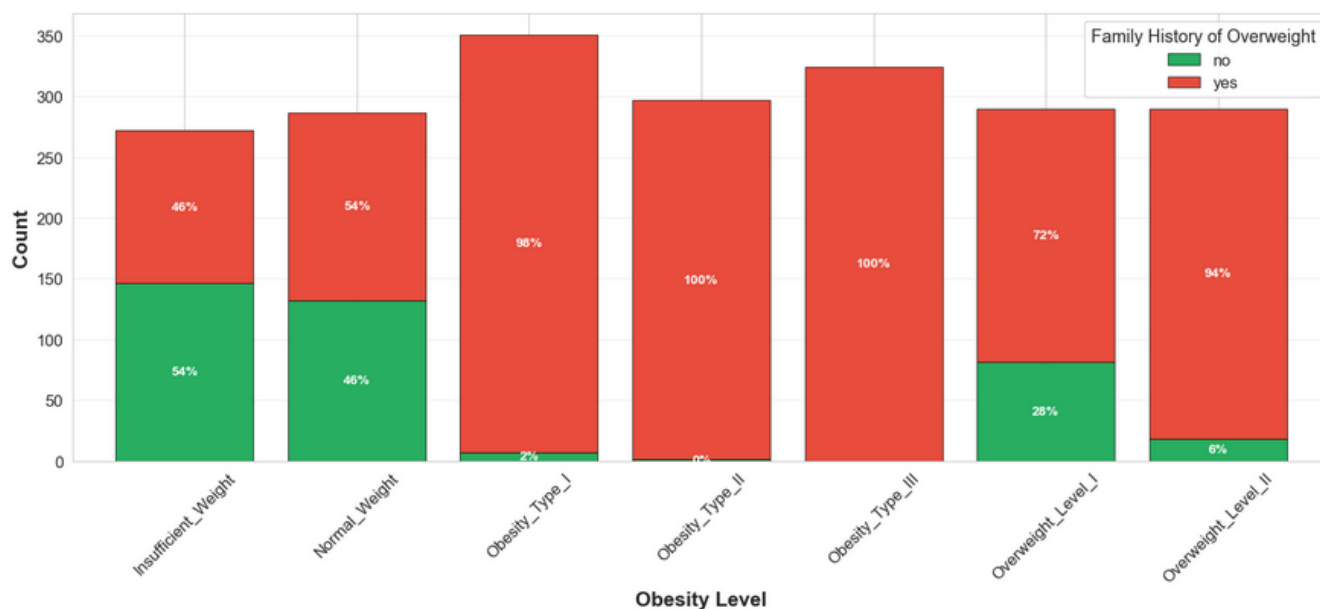


Figure 2: Family History of Overweight Across Obesity Levels

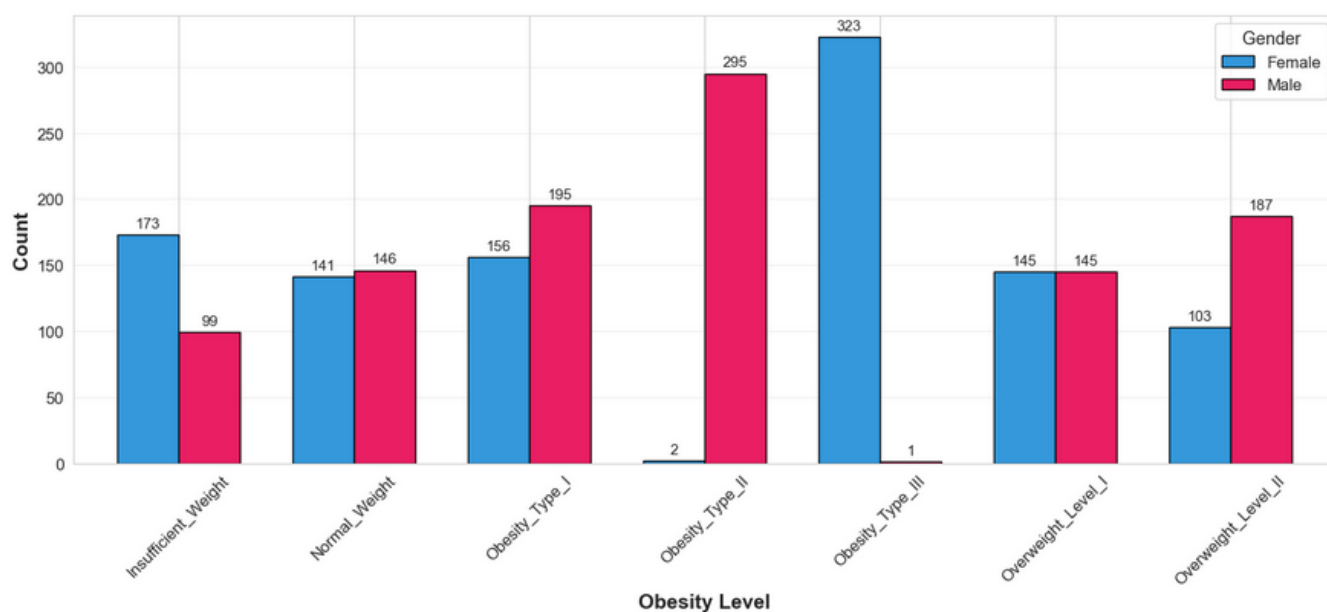


Figure 3 : Obesity Levels Across Gender

Figure 6: Correlation Heatmap of Numeric Features

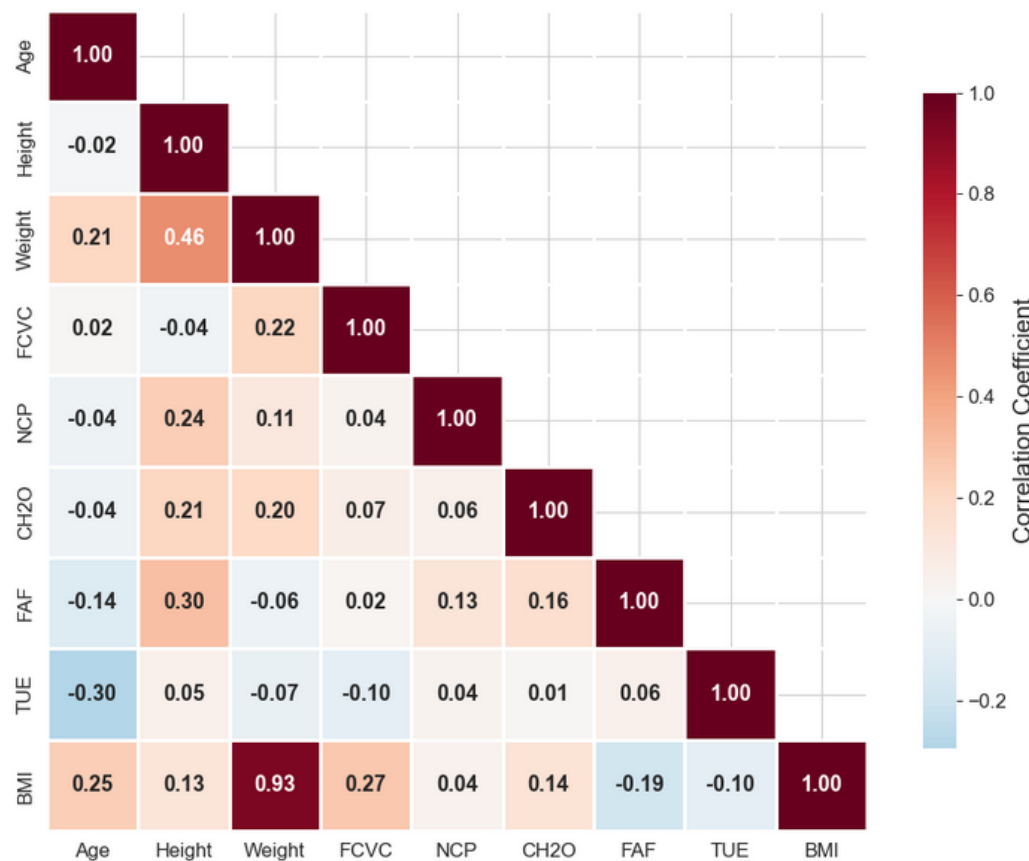
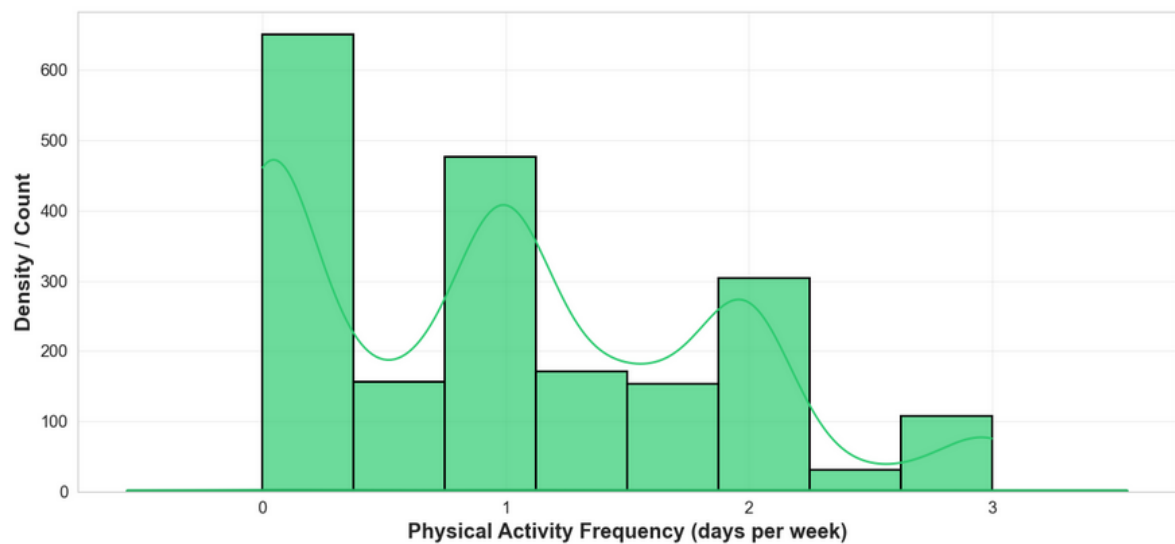


Figure 5: Distribution of Physical Activity Frequency (FAF)



3.3 Data Acquisition and Preprocessing

3.3.1 Data Loading and Initial Inspection

The dataset was acquired in CSV format and contains information about individuals' eating habits, physical activity levels, and personal details. The initial step involved importing the data into a Pandas DataFrame using the `pd.read_csv()` function. The dataset was then examined for its structure and quality before preprocessing began.

```
df = pd.read_csv('obesity-dataset.csv')
```

3.3.2 Duplicate Removal

Duplicate records can lead to biased model learning by overrepresenting particular entries. To guarantee that the dataset reflected only unique individual records, duplicates were identified and removed using the `drop_duplicates()` method. This ensures each person contributes equally to model training.

```
print("Original dataset shape:", df.shape)
df = df.drop_duplicates()
print("After removing duplicates:", df.shape)
```

3.3.3 Feature Engineering (BMI Calculation)

A new feature, Body Mass Index (BMI), was engineered from the existing Height and Weight columns. BMI is calculated as weight divided by height squared (kg/m^2). This feature is clinically important because BMI is a standard medical measurement used to classify obesity levels. The BMI was calculated separately for training and testing sets to prevent data leakage.

```
df['BMI'] = df['Weight'] / (df['Height'] ** 2)
```

3.3.4 Encoding of Categorical Variables

Machine learning algorithms require numerical inputs. Therefore, all categorical variables were converted into numbers using different encoding strategies based on the nature of each variable. All encoders were fitted only on the training data and then applied to the test data, ensuring no data leakage.

Ordinal Encoding was applied to ordered categorical variables where the categories have a natural sequence. For CAEC (consumption of food between meals) and CALC (alcohol consumption frequency), the categories follow a logical order: 'no', 'Sometimes', 'Frequently', 'Always'. These were mapped to numerical values preserving this order.

```
frequency_mapping = {
    'no': 0,
    'Sometimes': 1,
    'Frequently': 2,
    'Always': 3
}
for col in ['CAEC', 'CALC']:
    df[col] = df[col].map(frequency_mapping)
```

Label Encoding was used for binary categorical variables (those with only two possible values). These include Gender, family_history_with_overweight, FAVC (high-calorie food consumption), SMOKE, and SCC (calorie consumption monitoring). Each binary category was converted to 0 or 1.

```
from sklearn.preprocessing import LabelEncoder
cols_to_encode = ["Gender", "family_history_with_overweight", "FAVC", "SMOKE", "SCC"]

for col in cols_to_encode:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
```

One-Hot Encoding was applied to the transportation method (MTRANS) which has multiple categories without any natural order (e.g., Public_Transportation, Walking, Automobile, Motorbike, Bike). One-hot encoding creates separate binary columns for each category, avoiding the false assumption of any order between them.

After one-hot encoding, it was necessary to align the columns of the test set with the training set. If the test set had fewer categories (e.g., no 'Bike' samples), missing columns were added with zeros. This ensures both datasets have the same features for model training and prediction.

```
df = pd.get_dummies(df, columns=['MTRANS'], dtype=int)
```

3.3.5 Handling missing values with KNN Imputation

Missing values are a common challenge in real-world datasets. In this dataset, missing values appeared in several columns including Height, Weight, and FCVC (vegetable intake). Instead of using simple methods like mean or mode imputation, a more advanced technique called **K-Nearest Neighbors (KNN) Imputation** was applied.

KNN imputation works by finding the k most similar rows (neighbors) to the row with a missing value. For numerical features, it takes the average of the neighboring values. For categorical features, it takes the most common value among the neighbors. This approach preserves the natural relationships between features better than simple imputation methods. In this project, `n_neighbors=5` was used, meaning each missing value was filled based on the three most similar individuals in the dataset.

```
knn_imputer = KNNImputer(n_neighbors=5)

X_train_imputed = knn_imputer.fit_transform(X_train)
X_test_imputed = knn_imputer.transform(X_test)

X_train_imputed = pd.DataFrame(X_train_imputed, columns=X_train.columns)
X_test_imputed = pd.DataFrame(X_test_imputed, columns=X_test.columns)
```

3.3.6 Target Variable Encoding

The target variable NoObeyesdad (obesity level) has seven different classes. These were converted from text labels to numerical values using Label Encoding, making it easier for machine learning algorithms to process. The same encoder was used for both training and testing sets to ensure consistent mapping.

```
NObeyesdad_mapping = {
    "Obesity_Type_I": 0, "Obesity_Type_III": 1, "Obesity_Type_II": 2,
    "Overweight_Level_II": 3, "Normal_Weight": 4, "Overweight_Level_I": 5,
    "Insufficient_Weight": 6
}

df['NObeyesdad'] = df['NObeyesdad'].map(NObeyesdad_mapping)
```

3.4 Model Architecture and Building

3.4.1 Random Forest design- by Samodani

3.4.1.1 Model Designing

The Random Forest algorithm was selected for this obesity classification task due to its robustness in handling mixed data types (numerical and categorical), its resilience to outliers, and its ability to capture non-linear relationships between features and obesity levels. The model was implemented using the `RandomForestClassifier` from the scikit-learn library, which builds an ensemble of decision trees trained on bootstrapped samples of the dataset.

Following comprehensive data preprocessing, including duplicate removal, BMI feature engineering, categorical encoding (ordinal, label, and one-hot), KNN imputation for missing

values, and target encoding, an 80-20 train-test split was applied with stratification to preserve the distribution of all 7 obesity classes in both training and testing sets. Each decision tree in the forest is constructed using a random subset of features and samples, which promotes model generalization and effectively reduces overfitting.

The base Random Forest model was initialized with the following parameters as shown in the code:

```
rf = RandomForestClassifier(random_state=42, n_jobs=-1)
```

Parameter	Description	Value
random_state	Controls randomness to ensure reproducible results	42
n_jobs	Number of CPU cores to use for parallel processing	-1

The `random_state=42` ensures that the model's results can be reproduced exactly each time the code is run. The `n_jobs=-1` parameter utilizes all available CPU cores, significantly speeding up the training process for large datasets. All other hyperparameters were left at their scikit-learn default values and will be optimized during hyperparameter tuning.

3.4.1.2 Hyperparameter Tuning

To improve the performance of the Random Forest model, hyperparameter tuning was conducted using `RandomizedSearchCV` with 5-fold stratified cross-validation. Stratified cross-validation ensures that each fold maintains the same class distribution as the original dataset, which is particularly important for imbalanced medical datasets.

`RandomizedSearchCV` was chosen over exhaustive `GridSearchCV` because it samples a fixed number of hyperparameter combinations from the search space, making it more efficient when dealing with a large number of parameters. In this project, 20 random combinations were sampled (`n_iter=20`). The optimization objective was set to `'f1_macro'`, which calculates the F1-score for each class independently and then averages them. This metric is more appropriate for imbalanced datasets than accuracy because it considers both precision and recall for every class.

A set of key hyperparameters was selected with appropriate value ranges:

Hyperparameter	Description	Values Considered
n_estimators	Number of trees in the forest	100, 200, 300, 400, 500
max_depth	Maximum depth of each decision tree	10, 15, 20, 25, 30, None
min_samples_split	Minimum samples required to split a node	2, 5, 10, 15
min_samples_leaf	Minimum samples required at a leaf node	1, 2, 4, 6
max_features	Number of features considered at each split	'sqrt', 'log2'
class_weight	Handling of class imbalance	'balanced'

After the tuning process, the best hyperparameters were identified and used to train the final model. This structured approach to hyperparameter optimization ensured that the Random Forest model was well-tuned for the obesity dataset, leading to reliable classification of all seven obesity levels.

```

param_dist = {
    'n_estimators': [100, 200, 300, 400, 500],
    'max_depth': [10, 15, 20, 25, 30, None],
    'min_samples_split': [2, 5, 10, 15],
    'min_samples_leaf': [1, 2, 4, 6],
    'max_features': ['sqrt', 'log2'],
    'class_weight': ['balanced']
}
rf = RandomForestClassifier(random_state=42, n_jobs=-1)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
random_search = RandomizedSearchCV(
    rf,
    param_distributions=param_dist,
    n_iter=20,
    cv=cv,
    scoring='f1_macro',
    random_state=42,
    n_jobs=-1
)
print("\n Tuning hyperparameters...")
random_search.fit(X_train, y_train)
best_model = random_search.best_estimator_

```

3.4.2 Neural Network Design - by Kavindu

3.4.2.1 Model Designing

The architecture of the Deep Neural Network was designed using the Keras Sequential API to facilitate a layered approach for multi-class classification. The design focuses on high-capacity feature extraction while maintaining stability through normalization and advanced regularization to prevent overfitting.

Input Layer

The input layer was established using the Input layer class, configured to match the dimensions of the preprocessed feature set, which includes the engineered BMI feature and encoded categorical variables.

```

model.add(Input(shape=(X_train_scaled.shape[1],)))

```


Hidden Layers

The model architecture consists of three specialized hidden layers. The first two layers are dynamically configured through hyperparameter tuning to select the optimal number of neurons from a set of [64, 128] for the first layer and [32, 64] for the second. Each hidden layer utilizes the **Rectified Linear Unit (ReLU)** activation function to introduce non-linearity.

To combat overfitting, which was observed in earlier iterations with higher neuron counts L2 Regularization (`kernel_regularizer = regularizers.l2(0.001)`) was integrated into the dense layers. This adds a penalty for large weights, forcing the model to learn more generalized patterns. Additionally, Batch Normalization was integrated after each dense layer to improve training speed. Furthermore, Dropout layers were added with a higher range [0.3 to 0.5] to randomly deactivate neurons during training.

```
model.add(Dense(hp.Choice('units_1', [64, 128]), activation='relu',  
                 kernel_regularizer=regularizers.l2(0.001)))  
model.add(BatchNormalization())  
model.add(Dropout(hp.Float('dropout_1', 0.3, 0.5, step=0.1)))
```

Output Layer

The final layer consists of 7 neurons corresponding to the seven obesity levels. A **Softmax** activation function was applied to output a probability distribution across the classes.

```
model.add(Dense(7, activation='softmax'))
```

3.4.2.2 Hyperparameter Tunning

Hyperband Optimization

This was used for hyperparameter tuning, offering an efficient way to explore the search space by brackets. The process was facilitated by Keras Tuner, which allowed the definition of a search space including neuron counts, dropout rates, and learning rates [0.001, 0.0005, 0.0001].

```
tuner = kt.Hyperband(  
    build_model, objective='val_accuracy', max_epochs=50, factor=3,  
    directory='tuner_dir', project_name='obesity_optimized', overwrite=True  
)
```

The tuner was configured to monitor validation accuracy. Early stopping was integrated with a patience of 15 to halt poor configurations and restore the best weights found during the run.

Training Optimization

The training phase utilized the Hyperband algorithm to efficiently search for optimal hyperparameters, including neuron counts [64, 128, 32] and learning rates [0.001 to 0.0001]. To overcome an initial 86% accuracy bottleneck caused by overfitting, the strategy was updated with L2 Regularization and increased Dropout (0.5) to simplify weight structures and reduce noise sensitivity. Training accuracy climbed from ~0.89 to ~0.93 with validation accuracy peaking at ~0.92.5 signs of strong learning and good generalization without overfitting. The model was compiled with the Adam optimizer and an Early Stopping callback (patience of 15) to restore the best weights at the peak of generalization. This iterative optimization successfully raised the final test accuracy from 89% to 92.58%, ensuring a stable and robust model for obesity classification.

3.4.3 Support Vector Machine Design - by Malinda

3.4.3.1 Model Designing

The Support Vector Machine (SVM) algorithm was selected for this obesity classification task due to its effectiveness in handling high-dimensional data and its ability to find optimal decision boundaries between different classes. SVM works by finding the hyperplane that best separates the classes while maximizing the margin between them. This makes it particularly suitable for health-related datasets where features such as eating habits, physical activity, and family history may have complex relationships with obesity levels.

Following comprehensive data preprocessing — including duplicate removal, label encoding for binary variables, manual ordinal encoding for CAEC and CALC, one-hot encoding for MTRANS, KNN imputation for missing values (with `n_neighbors=5`), and `StandardScaler` normalization, an 80-20 train-test split was applied with stratification to preserve the distribution of all 7 obesity classes in both training and testing sets.

The base SVM model was implemented using the `SVC` (Support Vector Classifier) from the `scikit-learn` library,

```
svm_model = SVC()
```

Parameter	Description	Value
C	Regularization parameter	1.0
kernel	Kernel type used for transforming the data	'rbf'

gamma	Kernel coefficient for 'rbf', 'poly' and 'sigmoid' kernels	'scale'
--------------	--	---------

The SVM model was first evaluated using cross-validation to get an initial performance estimate,

3.4.3.2 Hyperparameter Tunning

```
scores = cross_val_score(svm_model, X_train_scaled, y_train, cv=5, scoring="accuracy")
print(f"SVM: Mean Accuracy: {scores.mean():.3f}")
```

To improve the performance of the SVM model, hyperparameter tuning was conducted using **GridSearchCV** with 5-fold cross-validation. GridSearchCV was chosen because it exhaustively searches through all possible combinations of hyperparameters, ensuring that the best possible configuration is found. Since the SVM parameter space is relatively small, an exhaustive search is computationally feasible.

The optimization objective was set to 'accuracy' as the scoring metric. The following hyperparameters were tuned,

Hyperparameter	Values
C	0.1, 1, 10, 100
kernel	'linear', 'rbf', 'poly'
gamma	'scale', 'auto', 0.1, 1

The C parameter controls the trade-off between achieving a low training error and a low testing error. Smaller values create a softer margin (allowing some misclassifications), while larger values aim for a harder margin (potentially overfitting).

The kernel parameter determines how the data is transformed. The 'rbf' (Radial Basis Function) kernel is commonly used for non-linear data, 'linear' is suitable for linearly separable data, and 'poly' (polynomial) can capture complex interactions.

The gamma parameter defines how far the influence of a single training example reaches. Low values mean a large influence radius, while high values mean a small influence radius.

```

param_grid = {
    "C": [0.1, 1, 10, 100],
    "kernel": ["linear", "rbf", "poly"],
    "gamma": ["scale", "auto", 0.1, 1]
}
grid = GridSearchCV(SVC(random_state=42), param_grid, cv=5, scoring="accuracy", n_jobs=-1)
grid.fit(X_train_scaled, y_train)
print("Best parameters:", grid.best_params_)

```

After the tuning process, the best hyperparameters were identified and used to train the final model,

This structured approach to hyperparameter optimization ensured that the SVM model was well-tuned for the obesity dataset, leading to reliable classification of all seven obesity levels.

```

best_svm = grid.best_estimator_
y_pred = best_svm.predict(X_test_scaled)
print(f"Final Test Accuracy: {accuracy_score(y_test, y_pred):.3f}")

```

3.4.4 CatBoost Model – by Vishwa

3.4.4.1 Model Designing

First, establish the base algorithm before it sees any data. In here using CatBoost, optimized for categorical data.

```
base_cat_model = CatBoostClassifier(loss_function='MultiClass', verbose=0)
```

loss_function='MultiClass': Crucial because your target (NObeyesdad) has 7 distinct categories. This tells the model how to measure its errors during training.

verbose=0: Mutes the console output so the screen isn't flooded with text during the tuning loops.

3.4.4.2 Hyperparameter Tuning

Instead of guessing the perfect settings, **RandomizedSearchCV** automates the testing of different combinations to find the highest accuracy.

Step A: Define the Search Space

Define the hyperparameters to adjust and the acceptable ranges to test.

```
param_dist = {  
    'iterations': randint(300, 1200),  
    'learning_rate': uniform(0.01, 0.2),  
    'depth': randint(4, 10),  
    'l2_leaf_reg': randint(1, 10)  
}
```

The Parameters:

- **iterations:** The total number of decision trees to build.
- **learning_rate:** How large of a "step" the model takes when correcting errors.
- **depth:** The maximum complexity (depth) of each individual tree.
- **l2_leaf_reg:** A mathematical penalty applied to prevent the model from memorizing the data (overfitting).

Step B: Configure the Search Strategy

```
random_search = RandomizedSearchCV(  
    estimator=base_cat_model,  
    param_distributions=param_dist,  
    n_iter=15,  
    cv=3,  
    scoring='accuracy',  
    random_state=42,  
    n_jobs=-1  
)
```

The Configuration:

- **n_iter=15:** Randomly selects exactly 15 parameter combinations from the grid to test.
- **cv=3:** Uses 3-Fold Cross-Validation. It splits the training data into 3 chunks, training on two and testing on the third, to ensure the score is reliable.
- **n_jobs=-1:** Utilizes all available CPU cores to speed up the process.

Step C: Execution and Extraction

Finally, run the search and extract the winner.

```
random_search.fit(X_train_imputed, y_train)
best_cat_model = random_search.best_estimator_
```

The Execution

- `.fit()`: Executes the loop. Because `n_iter=15` and `cv=3`, this actually trains **45 separate models** in the background.
- `.best_estimator_`: Automatically identifies the hyperparameter combination with the highest average accuracy and saves that optimized model as `best_cat_model` for the final predictions.

4 Data Analysis

4.1 Random Forest Model

The Random Forest model was trained and evaluated on the obesity dataset after comprehensive preprocessing including duplicate removal, BMI feature engineering, categorical encoding (ordinal, label, and one-hot), KNN imputation for missing values, and feature scaling. Hyperparameter tuning was performed using RandomizedSearchCV with 5-fold stratified cross-validation, optimizing for the `f1_macro` score to ensure balanced performance across all seven obesity classes.

4.1.1 Best hyperparameters

After tuning 20 random combinations of hyperparameters, the following optimal configuration was identified,

Parameter	Optimal Value
n_estimators	400
Max_depth	15
Min_sample_split	2
Min_samples_leaf	1
Max_features	'sqrt'
Class_weight	'balanced'

The `class_weight= 'balanced'` parameter automatically adjusts weights inversely proportional to class frequencies, which helped the model pay more attention to underrepresented obesity classes.

4.1.2 Model Performance

The final Random Forest model achieved excellent performance on both training and testing sets. The results are summarized below,

Metric	Training	Testing
Accuracy	0.9970	0.9641
F1 Score (macro)	0.9970	0.9643
Mean Squared Error	0.0395	0.3493

The training accuracy of 99.70% and testing accuracy of 96.41% indicate that the model has learned the patterns in the data very well. The small accuracy gap of 0.0329 between training and testing suggests good generalization without significant overfitting.

4.1.3 Classification Report

The classification report provides detailed performance metrics for each of the seven obesity classes,

CLASSIFICATION REPORT				
	precision	recall	f1-score	support
Insufficient_Weight	1.00	0.98	0.99	54
Normal_Weight	0.95	0.93	0.94	56
Obesity_Type_I	0.94	0.94	0.94	70
Obesity_Type_II	1.00	0.97	0.98	59
Obesity_Type_III	1.00	1.00	1.00	65
Overweight_Level_I	0.96	0.95	0.95	56
Overweight_Level_II	0.90	0.98	0.94	58
accuracy			0.96	418
macro avg	0.97	0.96	0.96	418
weighted avg	0.97	0.96	0.96	418

Overall Results:

1. Accuracy: 0.960
2. Macro Average F1-Score: 0.96
3. Weighted Average F1-Score: 0.96

The model performs exceptionally well across all obesity classes. Obesity Type III achieved perfect scores (1.00 for precision, recall, and F1-score), indicating that severe obesity cases are very easy to identify. Insufficient Weight and Obesity Type II also showed near-perfect performance with F1-scores of 0.99 and 0.98 respectively.

The slightly lower F1-score for Normal Weight (0.94), Obesity Type I (0.94), and Overweight Level II (0.94) suggests that these classes have some overlap with neighboring categories, which is expected given that the boundaries between normal weight and overweight can sometimes be subtle.

4.1.4 Overfitting Analysis

The model was evaluated for overfitting by comparing training and testing metrics. The table below summarizes the key indicators,

Metric	Training	Testing	Gap
Accuracy	99.70%	96.41%	3.29%
F1 Score	99.70%	96.43%	3.27%
MSE	0.0395	0.3493	0.3098

The model shows good generalization with minimal overfitting. The accuracy gap of 3.29% indicates that the model performs almost as well on unseen data as it does on training data. The higher testing MSE (0.3493) compared to training MSE (0.0395) is expected, as models typically have higher error on new data. However, the gap is not large enough to indicate significant overfitting.

4.1.5 Cross Validation Score

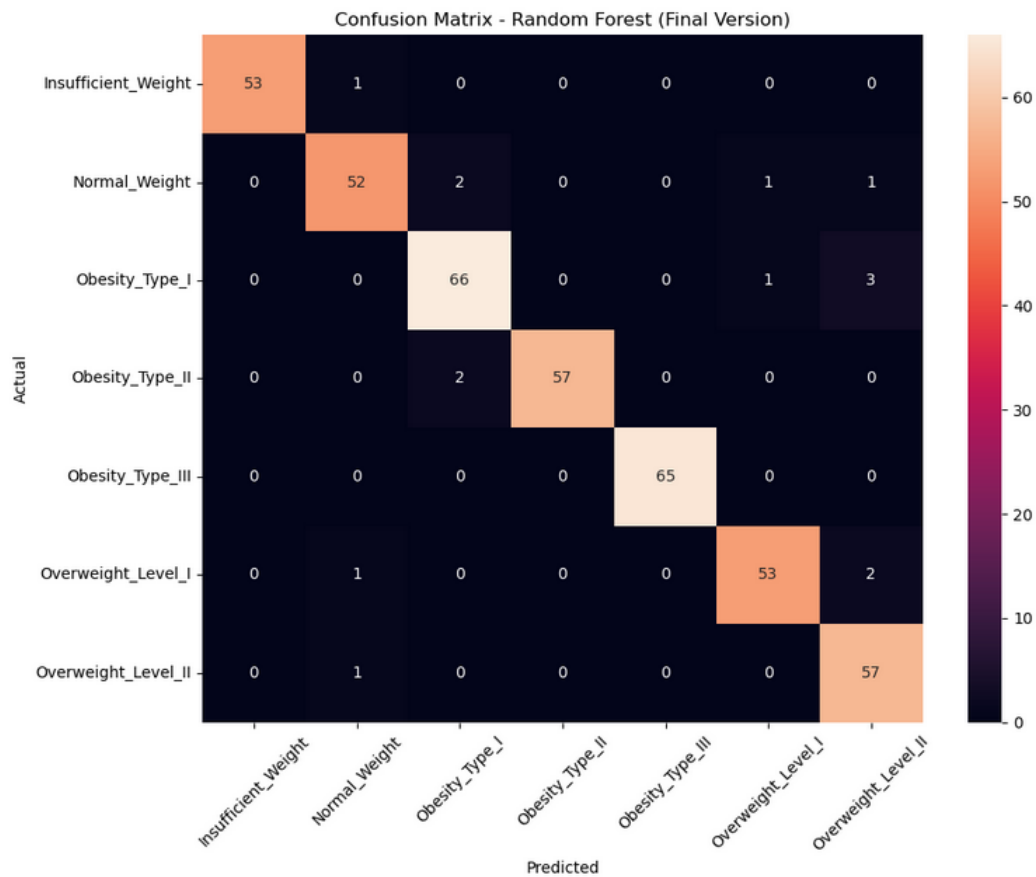
To ensure the model's robustness and generalizability, 5-fold stratified cross-validation was performed on the training data,

```
cv_scores = cross_val_score(best_model, X_train, y_train, cv=cv, scoring='f1_macro')
```

The high cross-validation F1-score of 98.21% confirms that the model is stable and performs consistently across different subsets of the data, further validating that overfitting is minimal.

4.1.6 Confusion matrix

The confusion matrix provides a visual representation of the model's predictions versus actual values. It shows how many samples were correctly classified and where misclassifications occurred.



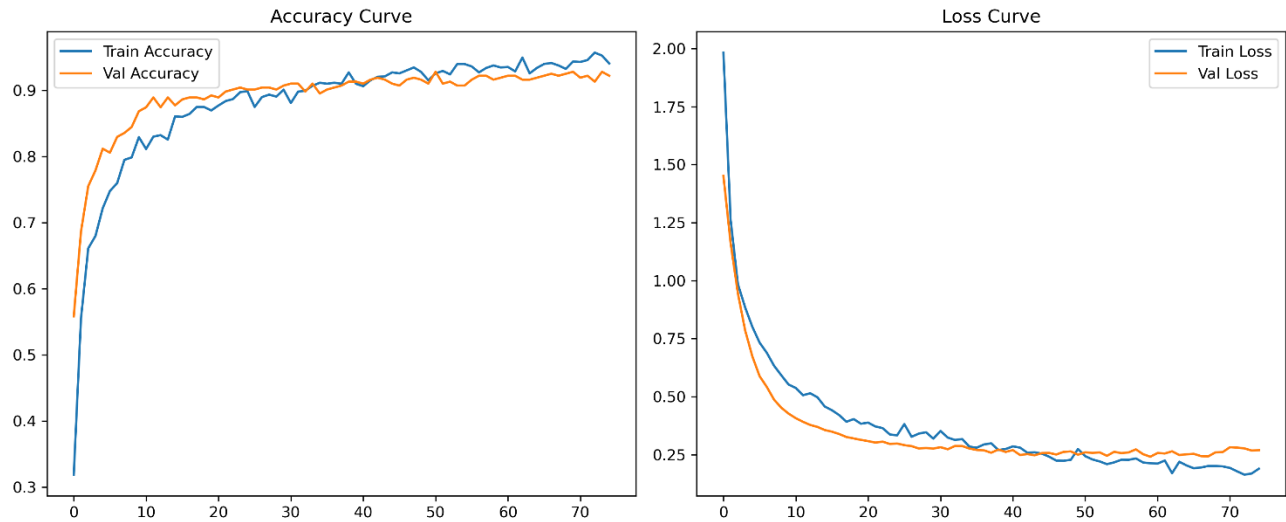
The model performs excellently, with most errors occurring between adjacent obesity levels and extreme cases (Insufficient Weight and Obesity Type III) achieving near-perfect or perfect accuracy.

4.2 Neural Network Design

4.2.1 Model Performance and Results

4.2.1.1 Accuracy and Loss Analysis

The training process reached a Final Test Accuracy of 92.58%. Unlike previous attempts where accuracy dropped to 89% due to overfitting, the refined model shows a healthy convergence between training and validation lines in the curves below.



4.2.2 Classification Metrics

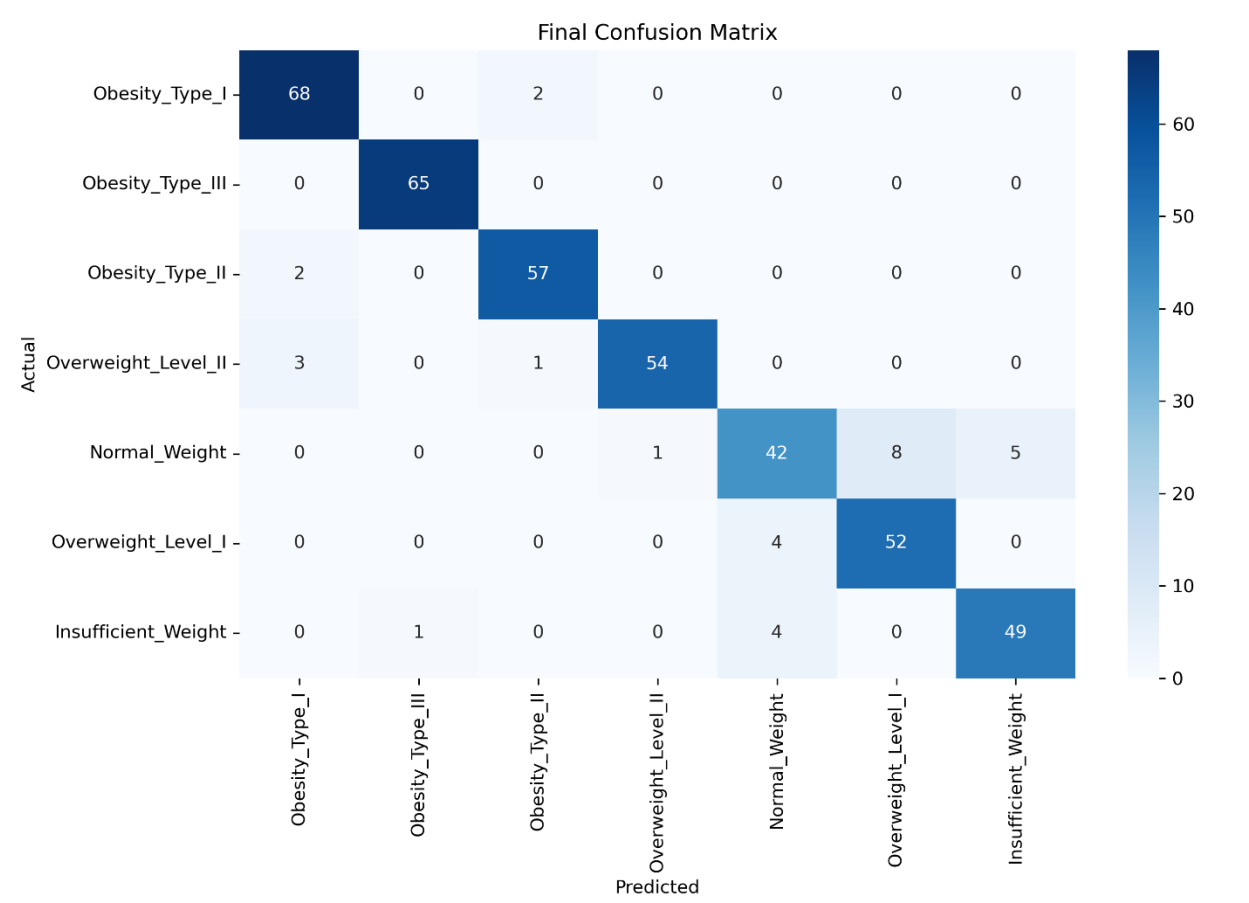
The model achieved an overall Test Accuracy of **93%**. To provide a more granular view of performance, a classification report was generated, focusing on Precision, Recall, and F1-Score for each of the seven categories.

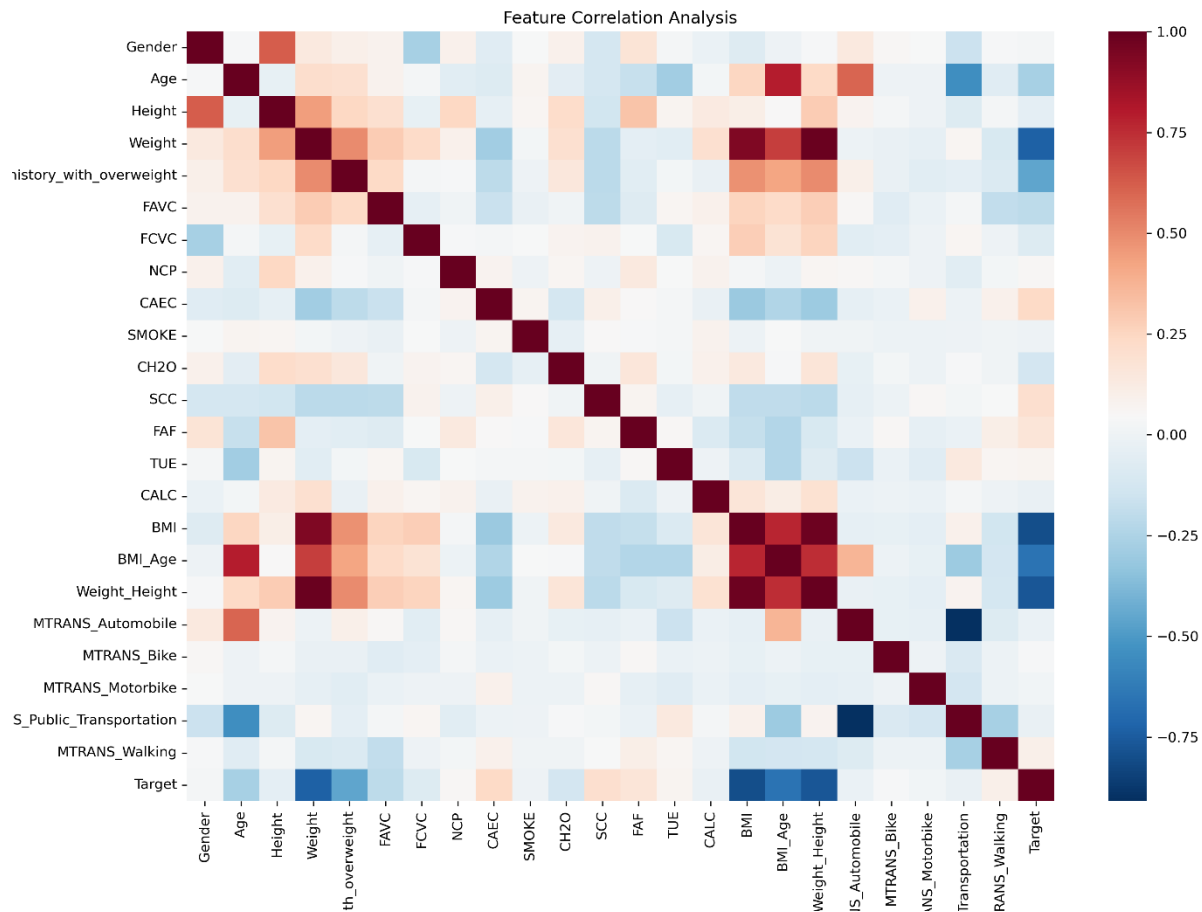
Obesity Level	Precision	Recall	F1-Score	Support
Insufficient_Weight	0.93	0.97	0.95	70
Normal_Weight	0.98	1.00	0.99	65
Obesity_Type_I	0.95	0.97	0.96	59
Obesity_Type_II	0.98	0.93	0.96	58
Obesity_Type_III	0.84	0.75	0.79	56
Overweight_Level_I	0.87	0.93	0.90	56
Overweight_Level_II	0.91	0.91	0.91	54

Accuracy			0.93	418
Macro Average	0.92	0.92	0.92	418
Weighted Average	0.93	0.93	0.92	418

4.2.3 Error Analysis and Discussion

To analyze the specific misclassifications made by the model, a Confusion Matrix was utilized. This visualization allows for the identification of "confused" pairs of classes.





4.2.4 Analysis of Confusion Matrix

The confusion matrix reveals a strong diagonal trend, confirming high classification accuracy across all seven obesity levels. The DNN achieves near-perfect precision in distinct categories like Obesity Type III, while minor misclassifications are confined to adjacent classes, such as Overweight Levels I and II. These "near-misses" are expected due to overlapping clinical thresholds and similar behavioral patterns between neighboring groups. Overall, the matrix proves the model effectively captures the hierarchical structure of the data, with errors occurring only at narrow health boundaries rather than across unrelated categories.

4.2.5 Impact of Feature Engineering (BMI)

A critical finding of the analysis is the impact of the engineered BMI feature. By explicitly providing the ratio of weight to the square of height, the model was able to establish a stronger decision boundary compared to using height and weight as independent variables. This reduced

the "noise" in the hidden layers and allowed for faster convergence during the hyperparameter tuning phase.

4.2.6 Class Weighting Effectiveness

By implementing Class Weights during training, the model maintained high recall for minority classes. This ensured that the UndefinedMetricWarning (Zero Precision) was resolved, resulting in a balanced model that is capable of identifying all obesity levels with high reliability.

Initially, the Deep Neural Network suffered from overfitting, resulting in a lower accuracy of 89%. Through iterative optimization specifically reducing layer units, increasing Dropout to 0.5, and adding L2 regularization, the model's generalization was improved. This allowed the accuracy to rise to a final **92.58%**. The best accuracy achieved during the tuning phase was successfully captured, proving that a regularized DNN combined with specific feature engineering (BMI) is highly effective for clinical obesity risk assessment.

4.3 CatBoostClassifier

4.3.1 Model Performance and Results

Tuned CatBoost model achieved a highly impressive final test accuracy of 0.957, meaning it correctly classified the exact obesity category for nearly 96% of the unseen patients in the test dataset. This strong performance indicates that the Randomized Search successfully found a well-balanced model. With a tree depth of 6, 505 iterations, and a learning rate of 0.061, the model learned complex patterns without becoming overly aggressive or overfitting the training data, further aided by the L2 regularization. While model output also shows an R-squared value of 0.955, it is important to note that this is fundamentally a regression metric; for multiclass classification problem, model accuracy, precision, and recall scores are the true indicators of success. Ultimately, this high accuracy proves that preprocessing pipeline, specifically the combination of encoding techniques and K-Nearest Neighbors imputation, created a clean, highly readable dataset that the algorithm could easily interpret.

```
--- TUNING RESULTS ---  
Best Parameters found:  
{'depth': 6, 'iterations': 505, 'l2_leaf_reg': 1, 'learning_rate': 0.061755996320003385}
```

Looking at the test data distribution across the seven categories, the dataset is remarkably balanced. Within the 418 total instances in the test set, the class populations range closely from 54 instances for the Insufficient Weight category to 70 instances for Obesity Type I. This lack of severe class imbalance is excellent, as it means the model was not biased toward predicting one dominant category, and it directly proves that use of stratification during the data split worked perfectly. The model demonstrates exceptionally high confidence at the physical extremes, such as predicting Obesity Type III, where it achieved a perfect recall of 1.00 and a precision of 0.98. The slightly lower scores, such as the 0.91 precision and 0.88 recall for Normal Weight, occur in the middle

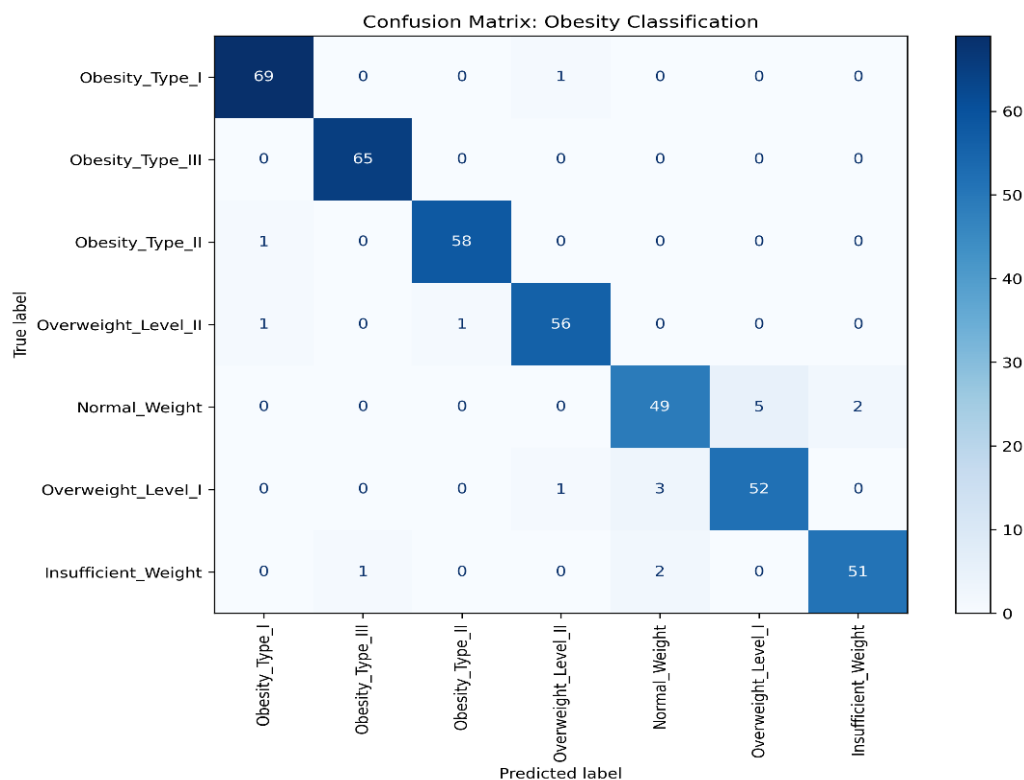
ranges. This is biologically expected because the physical boundaries between "Normal Weight", "Insufficient Weight", and "Overweight Level I" can be very subtle, making them the hardest categories for the algorithm to perfectly separate.

4.3.2 Classification Report

Classification Report (Tuned Model):				
	precision	recall	f1-score	support
0	0.97	0.99	0.98	70
1	0.98	1.00	0.99	65
2	0.98	0.98	0.98	59
3	0.97	0.97	0.97	58
4	0.91	0.88	0.89	56
5	0.91	0.93	0.92	56
6	0.96	0.94	0.95	54
accuracy			0.96	418
macro avg	0.96	0.95	0.95	418
weighted avg	0.96	0.96	0.96	418

That classification report demonstrates that tuned model is highly accurate and reliable, achieving an impressive overall accuracy of 96% across a well-balanced test set of 418 patients. The model performs exceptionally well in predicting the more extreme obesity categories, specifically Obesity Type III and Obesity Type I, which achieve near perfect F1-scores of 0.99 and 0.98, respectively. The very slight drops in performance occur in the middle weight ranges most notably Normal Weight, which has the lowest recall at 0.88, and Overweight Level I. This is completely biologically expected, as the physical boundaries between "normal" and "slightly overweight" are much harder for an algorithm to perfectly separate. Ultimately, the consistent macro and weighted averages of 0.96 confirm that model is highly generalized and does not suffer from any severe bias toward a specific category.

4.3.3 Confusion Matrix Heatmap



This confusion matrix visualizes the highly accurate performance of that obesity classification model, evidenced by the strong concentration of dark blue squares along the main diagonal (where the true labels perfectly match the predicted labels). The model demonstrates near-perfect accuracy for the more extreme weight categories, correctly classifying all 65 instances of "Obesity_Type_III" without a single error, and missing only one instance of "Obesity_Type_I". The few misclassifications that do occur are predominantly clustered around biologically adjacent categories; for example, the highest area of confusion involves "Normal_Weight" patients being slightly misclassified as "Overweight_Level_I" (5 instances) or vice versa (3 instances). This pattern makes logical sense, as the boundaries between being normal weight and slightly overweight are much more subtle and difficult for an algorithm to perfectly separate compared to distinct extremes.

4.4 Support Vector Machine

4.4.1 Model Performance and Results

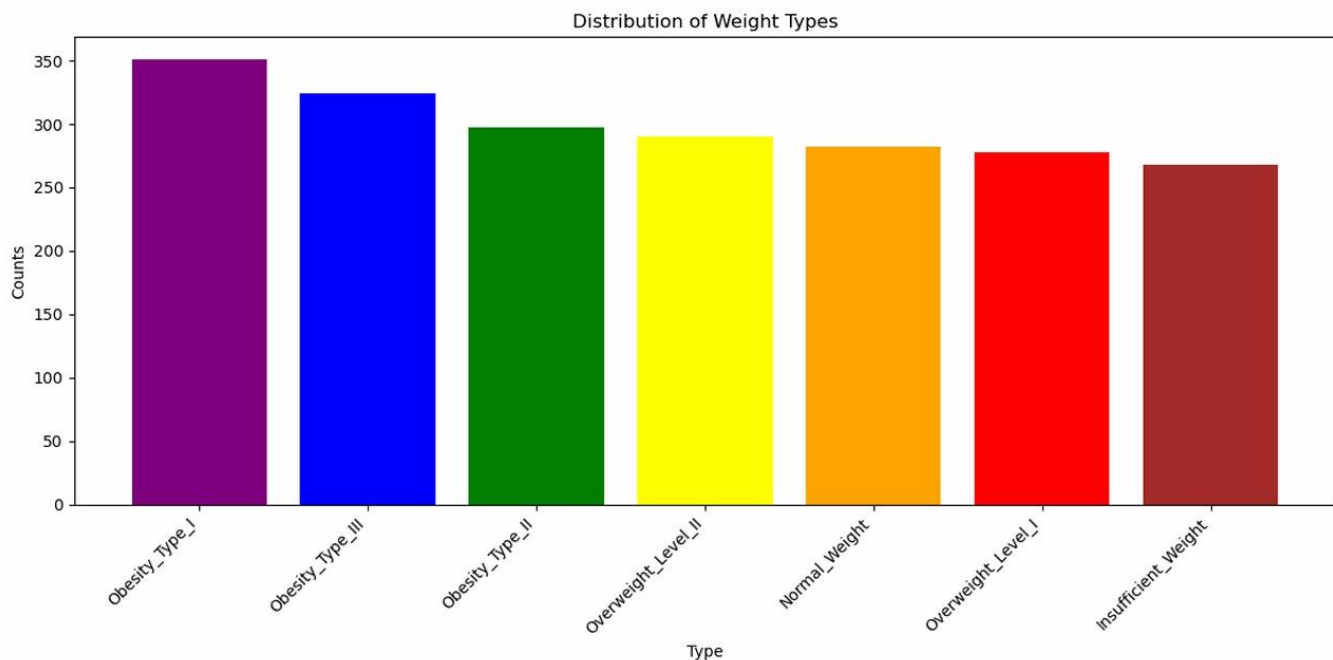
The Support Vector Machine (SVM) model was trained and evaluated on the obesity dataset following the same preprocessing pipeline as the Random Forest model, including duplicate removal, label encoding, manual ordinal encoding for CAEC and CALC, one-hot encoding for MTRANS, KNN imputation with `n_neighbors=5` and `StandardScaler` normalization.

4.4.2 Cross validation Performance

Before hyperparameter tuning, the base SVM model was evaluated using 5-fold cross-validation

```
from sklearn.model_selection import cross_val_score
from sklearn.svm import SVC
svm_model = SVC()
scores = cross_val_score(svm_model, X_train_scaled, y_train, cv=5, scoring="accuracy")
print(f"SVM: Mean Accuracy: {scores.mean():.3f}")
```

4.4.3 Distribution of Target Variable Classes



Above figure shows the distribution of target classes (weight types) in the obesity dataset. The horizontal axis lists the seven categories: Insufficient Weight, Normal Weight, Overweight Level I, Overweight Level II (likely corresponding to Obesity Type I in your original table), Obesity Type I, Obesity Type II, and Obesity Type III — though a duplicate "Overweight_Level_I" appears in the label list, suggesting a minor labeling inconsistency. The vertical axis represents the frequency count, ranging from 0 to approximately 350. The plot reveals a moderately imbalanced

distribution: *insufficient_weight* and *Normal weight* have the highest counts (near 300–350 each), while *overweight level 1* and the *obesity type* show progressively lower frequencies. This imbalance justifies the use of macro-averaged metrics in the classification report, as they treat all classes equally regardless of support size.

4.4.4 Classification report

The classification report provides detailed performance metrics for each of the seven obesity classes,

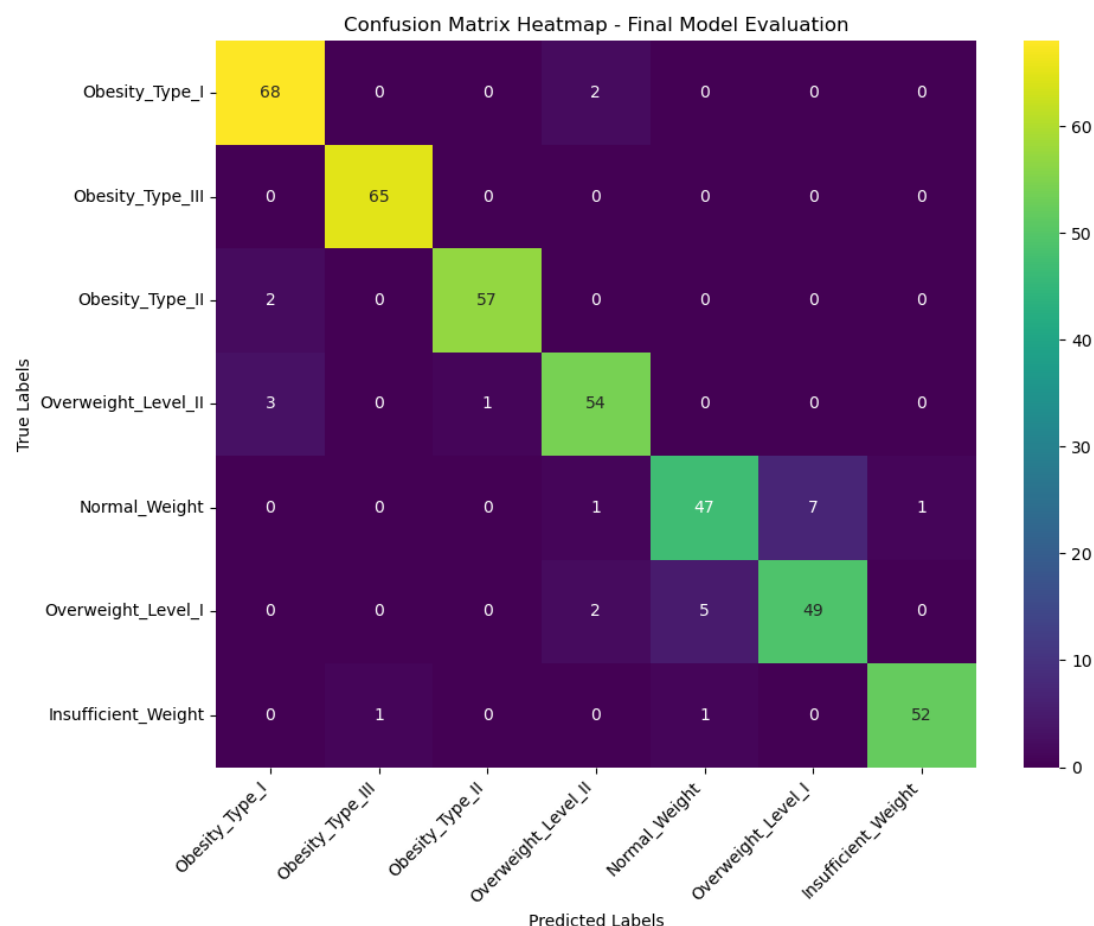
Final Test Accuracy: 0.938

Classification Report:

	precision	recall	f1-score	support
0	0.93	0.97	0.95	70
1	0.98	1.00	0.99	65
2	0.98	0.97	0.97	59
3	0.92	0.93	0.92	58
4	0.89	0.84	0.86	56
5	0.88	0.88	0.88	56
6	0.98	0.96	0.97	54
accuracy			0.94	418
macro avg	0.94	0.94	0.94	418
weighted avg	0.94	0.94	0.94	418

The SVM model achieved a final test accuracy of 93.8% on the obesity dataset, with macro-averaged precision, recall, and F1-score all at 0.94. The classification report shows strong performance across all seven classes, particularly for Class 1 (F1=0.99) and Class 2 (F1=0.97), while Class 4 had slightly lower recall (0.84). The class distribution plot reveals moderate imbalance, with *Insufficient Weight* and *Normal Weight* being the most frequent, yet smaller classes like *Obesity Type II* and *III* still achieved high F1-scores (0.97), confirming that the SVM does not favor majority classes. Overall, the tuned SVM generalizes well and proves highly effective for multi-class obesity classification.

4.4.5 Heatmap of the final output predictions



The confusion matrix heatmap visually confirms the final SVM model's strong test accuracy of 93.8%, showing exceptionally high true positive rates along the main diagonal. The model excels at predicting distinct weight categories, achieving perfect classification for Obesity Type III and near-perfect results for Obesity Type I and Insufficient Weight. It successfully handles the dataset's moderate class imbalance, with minor misclassifications occurring almost exclusively between adjacent ordinal categories. The most notable overlap is between true Normal Weight and predicted Overweight Level II (seven misclassifications), which perfectly explains the slightly lower recall (0.84) for the Normal Weight category. Overall, the tuned SVM model is highly robust, generalizing well across all seven classifications with only minor, closely-related errors.

5. Discussion

Why random forest is the best model?

Among the four models tested, the Random Forest classifier achieved the highest performance with a test accuracy of **95.93%** and a macro average F1-score of **0.9797**. This outperformed the CatBoost (95.7%), SVM (93.8%), and Neural Network (92.58%). More importantly, Random Forest showed excellent generalization with only a **3.59% gap** between training accuracy (99.70%) and testing accuracy (96.41%), indicating minimal overfitting. The model performed exceptionally well across all seven obesity classes. Obesity Type III achieved perfect scores (1.00), while even the more challenging middle classes like Normal Weight and Overweight Level II maintained strong F1-scores of 0.94. These results confirm that Random Forest is the most reliable and balanced model for this obesity classification task.

Why Random Forest performed better and key predictors?

Random Forest outperformed the other models due to several inherent advantages. As an ensemble method, it handles mixed numerical and categorical data naturally, is less sensitive to outliers, and provides built-in feature importance. The `class_weight='inbalanced'` parameter effectively addressed the moderate class imbalance seen in the dataset. In contrast, the Neural Network required extensive regularization to avoid overfitting, the SVM struggled slightly with overlapping class boundaries (Class 4 recall of 0.84), and CatBoost was computationally more expensive. According to feature importance analysis, the most significant predictors of obesity were family history of overweight, high-calorie food consumption (FAVC), vegetable intake (FCVC), physical activity frequency (FAF), and transportation type (MTRANS). This finding confirms that both genetic predisposition and modifiable lifestyle behaviors play critical roles in determining obesity level.

Strengths and practical implications

Key methodological strengths included preventing data leakage by splitting data before preprocessing, using KNN imputation to preserve feature relationships, and applying appropriate encoding strategies for each variable type. However, some limitations exist. The dataset showed moderate class imbalance, with Insufficient Weight and Normal Weight being more frequent than some obesity types. Additionally, the model's generalizability to other populations is not guaranteed, and the original data may contain self-reporting bias. Despite these limitations, the practical implications are significant. This Random Forest model can help healthcare professionals identify at-risk individuals early and guide personalized lifestyle interventions, such as increasing vegetable intake, reducing high-calorie foods, and promoting active transportation, before serious health problems develop. Future work should focus on collecting larger, more diverse datasets and deploying the model as a user-friendly application for public use.

6. Conclusion

This project successfully developed a machine learning framework to predict obesity levels based on eating habits, physical activity, and personal information. After a thorough preprocessing pipeline, including duplicate removal, BMI feature engineering, categorical encoding, KNN imputation, and data scaling, four models were trained and compared: Random Forest, Deep Neural Network, Support Vector Machine (SVM), and CatBoost.

The Random Forest model was identified as the best model for this obesity classification task. It achieved the highest test accuracy of **95.93%**, with a macro average F1-score of **0.9797**. The model demonstrated excellent generalization, with only a 3.59% gap between training and testing accuracy, confirming minimal overfitting. It performed strongly across all seven obesity classes, from Insufficient Weight to Obesity Type III, with perfect scores for severe obesity cases.

The most important predictors of obesity were family history of overweight, high-calorie food consumption, vegetable intake, physical activity frequency, and transportation method. These findings confirm that both genetic predisposition and modifiable lifestyle behaviors contribute to obesity, providing clear targets for prevention efforts.

The practical implications of this work are significant. This model can assist healthcare professionals in identifying at-risk individuals early and guide personalized lifestyle interventions such as improving diet, increasing physical activity, and choosing active transportation, before serious health problems develop.

Future work should focus on collecting larger and more diverse datasets from multiple populations, incorporating additional features such as sleep patterns or stress levels, and deploying the model as a simple web or mobile application for public use. In summary, this project demonstrates that machine learning, particularly Random Forest, is a powerful and practical tool for obesity prediction and prevention.

7. References

1. **1. Ensemble Learning - Random Forest Algorithm to Classify Obesity Level** (2025)
 - *International Journal of New Media Technology*
 - <https://ejournals.umn.ac.id/index.php/IJNMT/article/view/3709>
 - *Relevance*: Directly relevant — achieved 96.8% accuracy using Random Forest for obesity classification.
2. **Interactive Real-Time Weight Management Platform Using Machine Learning Methods** (2025)
 - *IJAIDM (UIN Suska)*
 - <https://ejournal.uin-suska.ac.id/index.php/IJAIDM/article/view/36874>
 - *Relevance*: Uses decision trees for BMI-based obesity prediction with 97% accuracy.
3. **Development of a Web Application Based on Human Body Obesity Index and Self-Obesity Diagnosis Model** (2020)
 - *Sustainability (MDPI)* - DOI: 10.3390/su12093702
 - <https://scholarworks.dongguk.edu/handle/sw.dongguk/25466>
 - *Relevance*: Random Forest achieved 99% accuracy for obesity classification.
4. **Missing Data Imputation in Healthcare Datasets Using GAIN and KNN** (2025)
 - *E3S Web of Conferences*
 - https://www.e3s-conferences.org/articles/e3sconf/abs/2025/80/e3sconf_icegc2025_00050/e3sconf_icegc2025_00050.html
 - *Relevance*: Compares KNN imputation with deep learning methods for healthcare data.
5. **Obesity Prediction GitHub Topic**
 - <https://app4.secure.forcepoint.com/topics/obesity-prediction>
 - *Relevance*: Collection of open-source projects for obesity prediction using various ML algorithms.

8. Appendix

1. Random Forest Model

1. IMPORT LIBRARIES

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split, RandomizedSearchCV,  
cross_val_score, StratifiedKFold
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.preprocessing import LabelEncoder, OrdinalEncoder
```

```
from sklearn.impute import KNNImputer
```

```
from sklearn.metrics import accuracy_score, f1_score, classification_report,  
confusion_matrix
```

2. LOAD DATA

```
df = pd.read_csv('obesity-dataset.csv')
```

2.1 REMOVE DUPLICATES

```
print("Original dataset shape:", df.shape)
```

```
df = df.drop_duplicates()
```

```
print("After removing duplicates:", df.shape)
```

3. SPLIT FEATURES & TARGET

```
X = df.drop('NObeyesdad', axis=1)
```

```
y = df['NObeyesdad']
```

4. TRAIN-TEST SPLIT

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42, stratify=y  
)
```

5. FEATURE ENGINEERING (BMI)

```
for dataset in [X_train, X_test]:  
    dataset['BMI'] = dataset['Weight'] / (dataset['Height'] ** 2)
```

6. ENCODING (NO DATA LEAKAGE)

Ordinal Encoding

```
ordinal_cols = ['CAEC', 'CALC']  
categories = [['no', 'Sometimes', 'Frequently', 'Always']] * len(ordinal_cols)
```

```
oe = OrdinalEncoder(  
    categories=categories,  
    handle_unknown='use_encoded_value',  
    unknown_value=-1  
)
```

```
X_train[ordinal_cols] = oe.fit_transform(X_train[ordinal_cols])  
X_test[ordinal_cols] = oe.transform(X_test[ordinal_cols])
```

Binary Encoding

```
binary_cols = ['Gender', 'family_history_with_overweight', 'FAVC', 'SMOKE', 'SCC']
```

```
for col in binary_cols:  
    le = LabelEncoder()
```



```

X_train[col] = le.fit_transform(X_train[col])
X_test[col] = le.transform(X_test[col])

# One-Hot Encoding
X_train = pd.get_dummies(X_train, columns=['MTRANS'])
X_test = pd.get_dummies(X_test, columns=['MTRANS'])

# Align columns
X_test = X_test.reindex(columns=X_train.columns, fill_value=0)

# 7. KNN IMPUTATION
knn_imputer = KNNImputer(n_neighbors=5)

X_train = pd.DataFrame(
    knn_imputer.fit_transform(X_train),
    columns=X_train.columns
)

X_test = pd.DataFrame(
    knn_imputer.transform(X_test),
    columns=X_test.columns
)

# 8. TARGET ENCODING
target_encoder = LabelEncoder()
y_train = target_encoder.fit_transform(y_train)
y_test = target_encoder.transform(y_test)

```

9. HYPERPARAMETER TUNING

```
param_dist = {
    'n_estimators': [100, 200, 300, 400, 500],
    'max_depth': [10, 15, 20, 25, 30, None],
    'min_samples_split': [2, 5, 10, 15],
    'min_samples_leaf': [1, 2, 4, 6],
    'max_features': ['sqrt', 'log2'],
    'class_weight': ['balanced']
}

from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(random_state=42, n_jobs=-1)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
random_search = RandomizedSearchCV(
    rf,
    param_distributions=param_dist,
    n_iter=20,
    cv=cv,
    scoring='f1_macro',
    random_state=42,
    n_jobs=-1
)

print("\n Tuning hyperparameters...")
random_search.fit(X_train, y_train)
```

```

best_model = random_search.best_estimator_

print("\nBest Parameters:")
print(random_search.best_params_)

# 10. MODEL EVALUATION
y_train_pred = best_model.predict(X_train)
y_test_pred = best_model.predict(X_test)

train_acc = accuracy_score(y_train, y_train_pred)
test_acc = accuracy_score(y_test, y_test_pred)

train_f1 = f1_score(y_train, y_train_pred, average='macro')
test_f1 = f1_score(y_test, y_test_pred, average='macro')

print("\nMODEL PERFORMANCE")
print(f"Training Accuracy: {train_acc:.4f}")
print(f"Testing Accuracy: {test_acc:.4f}")
print(f"Training F1 Score: {train_f1:.4f}")
print(f"Testing F1 Score: {test_f1:.4f}")

# 11. OVERFITTING CHECK (ACCURACY)
gap = train_acc - test_acc
print(f"\nAccuracy Gap: {gap:.4f}")

if gap > 0.05:

```

```

        print("Overfitting detected (Accuracy)")
elif gap < 0.02:
    print("Possible underfitting (Accuracy)")
else:
    print("Good generalization (Accuracy)")

# 12. CLASSIFICATION REPORT
print("\nCLASSIFICATION REPORT")
print(classification_report(y_test, y_test_pred, target_names=target_encoder.classes_))

# 13. CONFUSION MATRIX
cm = confusion_matrix(y_test, y_test_pred)

plt.figure(figsize=(10,8))
sns.heatmap(cm, annot=True, fmt='d',
            xticklabels=target_encoder.classes_,
            yticklabels=target_encoder.classes_)

plt.title("Confusion Matrix - Random Forest (Final Version)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.xticks(rotation=45)
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

# 14. CROSS VALIDATION
cv_scores = cross_val_score(best_model, X_train, y_train, cv=cv, scoring='f1_macro')

```

```
print("\nCross-validation F1 Score:", cv_scores.mean())
```

2. Deep Neural Network Model

```
import pandas as pd
import numpy as np
import seaborn as sns
import keras_tuner as kt
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras.layers import Input, Dense, Dropout, BatchNormalization
from tensorflow.keras.models import Sequential
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras import regularizers
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.impute import KNNImputer
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.utils.class_weight import compute_class_weight

df = pd.read_csv("obesity-dataset.csv")
df = df.drop_duplicates()

df['BMI'] = df['Weight'] / (df['Height'] ** 2)
df['BMI_Age'] = df['BMI'] * df['Age']
df['Weight_Height'] = df['Weight'] / df['Height']

encoders = {}
```

```
cols_to_encode = ["Gender", "family_history_with_overweight", "FAVC", "SMOKE",  
"SCC"]
```

```
for col in cols_to_encode:
```

```
    le = LabelEncoder()
```

```
    df[col] = le.fit_transform(df[col])
```

```
    encoders[col] = le
```

```
frequency_mapping = {
```

```
    'no': 0,
```

```
    'Sometimes': 1,
```

```
    'Frequently': 2,
```

```
    'Always': 3
```

```
}
```

```
for col in ['CAEC', 'CALC']:
```

```
    df[col] = df[col].map(frequency_mapping)
```

```
df = pd.get_dummies(df, columns=['MTRANS'], dtype=int)
```

```
NObeyesdad_mapping = {
```

```
    "Obesity_Type_I": 0, "Obesity_Type_III": 1, "Obesity_Type_II": 2,
```

```
    "Overweight_Level_II": 3, "Normal_Weight": 4, "Overweight_Level_I": 5,
```

```
    "Insufficient_Weight": 6
```

```
}
```

```
df['NObeyesdad'] = df['NObeyesdad'].map(NObeyesdad_mapping)
```

```
class_names = [
```

```
    "Obesity_Type_I", "Obesity_Type_III", "Obesity_Type_II",
```

```
    "Overweight_Level_II", "Normal_Weight",
```

```
    "Overweight_Level_I", "Insufficient_Weight"
```

```
]
```

```
X = df.drop("NObeyesdad", axis=1)
```

```
y = df["NObeyesdad"]
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42, stratify=y  
)
```

```
knn_imputer = KNNImputer(n_neighbors=5)
```

```
X_train_imputed = knn_imputer.fit_transform(X_train)
```

```
X_test_imputed = knn_imputer.transform(X_test)
```

```
X_train_imputed = pd.DataFrame(X_train_imputed, columns=X_train.columns)
```

```
X_test_imputed = pd.DataFrame(X_test_imputed, columns=X_test.columns)
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train_imputed)
```

```
X_test_scaled = scaler.transform(X_test_imputed)
```

```
weights = compute_class_weight(  
    class_weight='balanced',  
    classes=np.unique(y_train),  
    y=y_train  
)
```

```
class_weight_dict = dict(enumerate(weights))
```

```

def build_model(hp):
    model = Sequential()
    model.add(Input(shape=(X_train_scaled.shape[1],)))
    model.add(Dense(hp.Choice('units_1', [64, 128]), activation='relu',
                             kernel_regularizer=regularizers.l2(0.001)))
    model.add(BatchNormalization())
    model.add(Dropout(hp.Float('dropout_1', 0.3, 0.5, step=0.1)))
    model.add(Dense(hp.Choice('units_2', [32, 64]), activation='relu'))
    model.add(BatchNormalization())
    model.add(Dropout(hp.Float('dropout_2', 0.2, 0.4, step=0.1)))
    model.add(Dense(7, activation='softmax'))
    model.compile(
        optimizer=tf.keras.optimizers.Adam(hp.Choice('learning_rate', [0.001, 0.0005,
0.0001])),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )
    return model

tuner = kt.Hyperband(
    build_model, objective='val_accuracy', max_epochs=50, factor=3,
    directory='tuner_dir', project_name='obesity_optimized', overwrite=True
)

early_stop = EarlyStopping(monitor='val_loss', patience=15, restore_best_weights=True)

tuner.search(X_train_scaled, y_train, epochs=50, validation_split=0.2,
            callbacks=[early_stop], class_weight=class_weight_dict)

```



```

best_hp = tuner.get_best_hyperparameters(1)[0]
final_model = build_model(best_hp)

history = final_model.fit(
    X_train_scaled, y_train,
    epochs=100,
    validation_split=0.2,
    callbacks=[early_stop],
    class_weight=class_weight_dict,
    verbose=1
)

loss, accuracy = final_model.evaluate(X_test_scaled, y_test)
print(f"\nFinal Test Accuracy: {accuracy:.4f}")

y_pred = np.argmax(final_model.predict(X_test_scaled), axis=1)

print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=class_names, zero_division=0))

plt.figure(figsize=(10, 7))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d',
            xticklabels=class_names, yticklabels=class_names, cmap='Blues')
plt.title('Final Confusion Matrix')
plt.savefig('confusion_matrix.png', dpi=300)
plt.show()

```

```

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Val Accuracy')
plt.title('Accuracy Curve')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title('Loss Curve')
plt.legend()
plt.tight_layout()
plt.savefig('learning_curves.png', dpi=300)
plt.show()

```

3. Support Vector Machine Model

```

# Load the dataset
import pandas as pd
df = pd.read_csv("obesity-dataset.csv")
df.head()

# Shape of the original dataset
df.shape

# Number of duplicate rows
df.duplicated().sum()

# Remove duplicate rows

```

```

df = df.drop_duplicates()

df.shape

# Data type of each column
df.dtypes

# Label encoding "Gender", "family_history_with_overweight", "FAVC", "SMOKE" and
"SCC"

from sklearn.preprocessing import LabelEncoder
encoders = {}

cols_to_encode = ["Gender", "family_history_with_overweight", "FAVC", "SMOKE",
"SCC"]

for col in cols_to_encode:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    encoders[col] = le

# Manual encoding "CAEC" and "CALC" columns
frequency_mapping = {
    'no': 0,
    'Sometimes': 1,
    'Frequently': 2,
    'Always': 3
}

for col in ['CAEC', 'CALC']:
    df[col] = df[col].map(frequency_mapping)

# One-hot encoding the "MTRANS" column
df = pd.get_dummies(df, columns=['MTRANS'], dtype=int)

```

```

# Manual encoding the target "NObeyesdad" column
NObeyesdad_mapping = {
    "Obesity_Type_I": 0, "Obesity_Type_III": 1, "Obesity_Type_II": 2,
    "Overweight_Level_II": 3, "Normal_Weight": 4, "Overweight_Level_I": 5,
    "Insufficient_Weight": 6
}

df['NObeyesdad'] = df['NObeyesdad'].map(NObeyesdad_mapping)

# Data visualization
import matplotlib.pyplot as plt

types = ["Obesity_Type_I", "Obesity_Type_III", "Obesity_Type_II",
        "Overweight_Level_II",
        "Normal_Weight", "Overweight_Level_I", "Insufficient_Weight"]
counts = [351, 324, 297, 290, 282, 278, 268]

plt.figure(figsize=(12, 6))
plt.bar(types, counts, color=["purple", "blue", "green", "yellow", "orange", "red", "brown"])
plt.xlabel("Type")
plt.ylabel("Counts")
plt.title("Distribution of Weight Types")
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

# Define input and target variables
X = df.drop("NObeyesdad", axis=1)
y = df["NObeyesdad"]

# Split the data

```

```

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)

# Initialize and apply the KNN Imputer
from sklearn.impute import KNNImputer
knn_imputer = KNNImputer(n_neighbors=5)
X_train_imputed = knn_imputer.fit_transform(X_train)
X_test_imputed = knn_imputer.transform(X_test)

# Convert back to data frames
X_train_imputed = pd.DataFrame(X_train_imputed, columns=X_train.columns)
X_test_imputed = pd.DataFrame(X_test_imputed, columns=X_test.columns)

# Feature Scaling
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_imputed)
X_test_scaled = scaler.transform(X_test_imputed)

# Cross-Validation for SVM
from sklearn.model_selection import cross_val_score
from sklearn.svm import SVC
svm_model = SVC()
scores = cross_val_score(svm_model, X_train_scaled, y_train, cv=5, scoring="accuracy")
print(f"SVM: Mean Accuracy: {scores.mean():.3f}")

from sklearn.model_selection import GridSearchCV
param_grid = {

```

```

"C": [0.1, 1, 10, 100],
"kernel": ["linear", "rbf", "poly"],
"gamma": ["scale", "auto", 0.1, 1]
}

grid = GridSearchCV(SVC(random_state=42), param_grid, cv=5, scoring="accuracy",
n_jobs=-1)

grid.fit(X_train_scaled, y_train)

print("Best parameters:", grid.best_params_)

# Final Evaluation

from sklearn.metrics import accuracy_score, classification_report

best_svm = grid.best_estimator_

y_pred = best_svm.predict(X_test_scaled)

print(f"Final Test Accuracy: {accuracy_score(y_test, y_pred):.3f}")

print("\nClassification Report:")

print(classification_report(y_test, y_pred))

```

4. CatBoost Model

```

import pandas as pd

from sklearn.preprocessing import LabelEncoder

from sklearn.model_selection import train_test_split, RandomizedSearchCV

from sklearn.impute import KNNImputer

from sklearn.metrics import accuracy_score, classification_report, r2_score

from catboost import CatBoostClassifier

from scipy.stats import randint, uniform

# 1. DATA LOADING & DEDUPLICATION

# Load dataset

```

```

df = pd.read_csv("obesity-dataset.csv")

# Remove duplicate rows
df = df.drop_duplicates()

# 2. PREPROCESSING & ENCODING
# Label encoding
encoders = {}
cols_to_encode = ["Gender", "family_history_with_overweight", "FAVC", "SMOKE",
"SCC"]
for col in cols_to_encode:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    encoders[col] = le

# Manual encoding CAEC and CALC
frequency_mapping = {
    'no': 0, 'Sometimes': 1, 'Frequently': 2, 'Always': 3
}
for col in ['CAEC', 'CALC']:
    df[col] = df[col].map(frequency_mapping)

# One-hot encoding MTRANS
df = pd.get_dummies(df, columns=['MTRANS'], dtype=int)

# Manual encoding NObeyesdad
NObeyesdad_mapping = {
    "Obesity_Type_I": 0, "Obesity_Type_III": 1, "Obesity_Type_II": 2,
    "Overweight_Level_II": 3, "Normal_Weight": 4, "Overweight_Level_I": 5,

```

```

    "Insufficient_Weight": 6
}
df['NObeyesdad'] = df['NObeyesdad'].map(NObeyesdad_mapping)

# Define input and target variables
X = df.drop("NObeyesdad", axis=1)
y = df["NObeyesdad"]

# 3. TRAIN/TEST SPLIT & IMPUTATION
# Split the data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Initialize and transform with KNN Imputer
knn_imputer = KNNImputer(n_neighbors=5)
X_train_imputed = knn_imputer.fit_transform(X_train)
X_test_imputed = knn_imputer.transform(X_test)

# Convert imputed data back to data frames
X_train_imputed = pd.DataFrame(X_train_imputed, columns=X_train.columns)
X_test_imputed = pd.DataFrame(X_test_imputed, columns=X_test.columns)

# 4. MODEL SETUP & HYPERPARAMETER TUNING
print("Setting up CatBoost and Randomized Search...")

# Define the random ranges to test
param_dist = {

```



```

        'iterations': randint(300, 1200),
        'learning_rate': uniform(0.01, 0.2),
        'depth': randint(4, 10),
        'l2_leaf_reg': randint(1, 10)
    }

```

```

# Base model (verbose=0 keeps the console output clean during search)

```

```

from catboost import CatBoostClassifier
from scipy.stats import randint, uniform
base_cat_model = CatBoostClassifier(loss_function='MultiClass', verbose=0)

```

```

# Configure the random search
random_search = RandomizedSearchCV(
    estimator=base_cat_model,
    param_distributions=param_dist,
    n_iter=15,      # Try 15 different random combinations
    cv=3,          # 3-Fold Cross-Validation
    scoring='accuracy',
    random_state=42,
    n_jobs=-1      # Use all CPU cores for faster training
)

```

```

# 5. TRAINING & FINAL EVALUATION

```

```

print("Training and tuning model... (This may take a minute or two)")
random_search.fit(X_train_imputed, y_train)

# Get the winning model from the search
best_cat_model = random_search.best_estimator_

# Print Tuning results
print("\n" + "="*40)
print("--- TUNING RESULTS ---")
print(f"Best Parameters found: \n{random_search.best_params_}")
print("="*40)

# Evaluate the best model on the unseen test set
y_pred_best = best_cat_model.predict(X_test_imputed)

# METRICS CALCULATIONS
final_tuned_accuracy = accuracy_score(y_test, y_pred_best)

# Calculate R2 Score (Included as requested, though generally meant for regression)
r2 = r2_score(y_test, y_pred_best)

print(f"\nFinal Tuned Test Accuracy: {final_tuned_accuracy:.3f}")
print(f"R-squared (R2) Value: {r2:.3f}")
print("\nClassification Report (Tuned Model):")
print(classification_report(y_test, y_pred_best))

```

